XCS224R Assignment 4

Due Sunday, April 12 at 11:59pm PT.

Guidelines

1. If you have a question about this homework, we encourage you to post your question on our Slack channel, at http://xcs224r-scpd.slack.com/
2. Familiarize yourself with the collaboration and honor code policy before starting work.
3. For the environment setup for the coding problems, please refer to the `README.md` file in the assignment directory.
4. For the coding problems, you must use the packages specified in the provided environment description. Since the autograder uses this environment, we will not be able to grade any submissions which import unexpected libraries.

Submission Instructions

Written Submission: Some questions in this assignment require a written response. For these questions, you should submit a PDF with your solutions online in the online student portal. As long as the PDF is legible and organized, the course staff has no preference between a handwritten and a typeset LaTeX submission. If you wish to typeset your submission and are new to LaTeX, you can get started with the following:

- Type responses only in `submission.tex`.
- Submit the compiled PDF, **not** `submission.tex`.
- Use the commented instructions within the `Makefile` and `README.md` to get started.

Notebook Submission: Some questions in this assignment require notebook and Python code responses. For these questions, if you use Colab, you should run the `collect_submission.ipynb` notebook to generate a zip file of your code. If you use a local or Azure platform, you should run the `collect_submission.sh` script to generate a zip file of your code. When the zip file is generated, you should submit it to the Gradescope Coding submission for the corresponding assignment.

Inline Submission: Some questions in the notebook require you to provide inline answers. You should first write your answers in the notebook, and then submit them to the Gradescope Written Submission for the corresponding assignment.

Honor code

We strongly encourage students to form study groups. Students may discuss and work on homework problems in groups. However, each student must write down the solutions independently, and without referring to written notes from the joint session. In other words, each student must understand the solution well enough in order to reconstruct it by him/herself. In addition, each student should write on the problem set the set of people with whom s/he collaborated. Further, because we occasionally reuse problem set questions from previous years, we expect students not to copy, refer to, or look at the solutions in preparing their answers. It is an honor code violation to intentionally refer to a previous year's solutions. For SCPD classes, it is also important that students avoid opening pull requests containing their solution code on the shared assignment repositories. More information regarding the Stanford honor code can be found at https://communitystandards.stanford.edu/policies-and-guidance/honor-code.

Writing Code and Running the Autograder

All your code should be entered into the `src/submission/` directory. When editing files in `src/submission/`, please only make changes between the lines containing `### START CODE HERE ###` and `### END CODE HERE ###`. Do not make changes to files outside the `src/submission/` directory.

The unit tests in `src/grader.py` (the autograder) will be used to verify a correct submission. Run the autograder locally using the following terminal command within the `src/` subdirectory:

```
$ python grader.py
```

There are two types of unit tests used by the autograder:

- `basic`: These tests are provided to make sure that your inputs and outputs are on the right track, and that the hidden evaluation tests will be able to execute.
- `hidden`: These unit tests are NOT visible locally. These hidden tests will be run when you submit your code to the Gradescope autograder via the online student portal, and will provide feedback on how many points you have earned. These tests will evaluate elements of the assignment, and run your code with more complex inputs and corner cases. Just because your code passed the basic local tests does not necessarily mean that they will pass all of the hidden tests.

For debugging purposes, you can run a single unit test locally. For example, you can run the test case `3a-0-basic` using the following terminal command within the `src/` subdirectory:

```
$ python grader.py 3a-0-basic
```

Before beginning this course, please walk through the Anaconda Setup for XCS Courses to familiarize yourself with the coding environment. Use the env defined in `src/environment.yml` to run your code. This is the same environment used by the online autograder.

Test Cases

The autograder is a thin wrapper over the python `unittest` framework. It can be run either locally (on your computer) or remotely (on SCPD servers). The following description demonstrates what test results will look like for both local and remote execution. For the sake of example, we will consider two generic tests: `1a-0-basic` and `1a-1-hidden`.

Local Execution - Basic Tests

When a basic test like `1a-0-basic` passes locally, the autograder will indicate success:

```
----- START 1a-0-basic:  Basic test case.
----- END 1a-0-basic [took 0:00:00.000062 (max allowed 1 seconds), 2/2 points]
```

When a basic test like `1a-0-basic` fails locally, the error is printed to the terminal, along with a stack trace indicating where the error occurred:

```
----- START 1a-0-basic:  Basic test case.
<class 'AssertionError'>
{'a': 2, 'b': 1} != None
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/Users/grinch/Local_Documents/SCPD/XCS221/A1/src/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/Users/grinch/Local_Documents/Software/anaconda3/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
----- END 1a-0-basic [took 0:00:00.003809 (max allowed 1 seconds), 0/2 points]
```

This error caused the test to fail.

In this case, start your debugging in line 23 of grader.py.

Remote Execution

Basic and hidden tests are treated the same by the remote autograder, however the output of hidden tests will only appear once you upload your code to GradeScope. Here are screenshots of failed basic and hidden tests. Notice that the same information (error and stack trace) is provided as the in local autograder, now for both basic and hidden tests.

1a-0-basic) Basic test case. (0.0/2.0)

```
<class 'AssertionError'>:  {'a': 2, 'b': 1} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 23, in test_0
    submission.extractWordFeatures("a b a"))
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

Just like in the local autograder, this error caused the test to fail.

Just like in the local autograder, start your debugging in line 23 of grader.py.

1a-1-hidden) Test multiple instances of the same word in a sentence. (0.0/3.0)

```
<class 'AssertionError'>:  {'a': 23, 'ab': 22, 'aa': 24, 'c': 16, 'b': 15} != None
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 59, in testPartExecutor
    yield
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 605, in run
    testMethod()
  File "/autograder/source/graderUtil.py", line 54, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/graderUtil.py", line 83, in wrapper
    result = func(*args, **kwargs)
  File "/autograder/source/grader.py", line 31, in test_1
    self.compare_with_solution_or_wait(submission, 'extractWordFeatures', lambda f: f(sentence))
  File "/autograder/source/graderUtil.py", line 183, in compare_with_solution_or_wait
    self.assertEqual(ans1, ans2)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 829, in assertEqual
    assertion_func(first, second, msg=msg)
  File "/autograder/source/miniconda/envs/XCS221/lib/python3.6/unittest/case.py", line 822, in _baseAssertEqual
    raise self.failureException(msg)
```

This error caused the test to fail.

Start your debugging in line 31 of grader.py.

Finally, here is what it looks like when basic and hidden tests pass in the remote autograder.

1a-0-basic) Basic test case. (2.0/2.0)

1a-1-hidden) Test multiple instances of the same word in a sentence. (3.0/3.0)

Overview

This assignment consists of two parts.

In Part 1, we will be looking at goal-conditioned reinforcement learning and hindsight experience replay (HER). In particular, you will:

1. Adapt an existing model (Deep Q-Network) to be goal-conditioned.
2. Run goal-conditioned DQN on two environments.
3. Implement Hindsight Experience Replay (HER) [1] [2] on top of goal-conditioned DQN.
4. Compare the performance with and without HER.

In Part 2, we will be exploring meta-reinforcement learning algorithms. In particular, you will:

1. Experiment with a black-box meta-RL method [3] trained end-to-end.
2. Implement components of DREAM [4] to replace the end-to-end optimization objective.
3. Compare the performance of end-to-end versus decoupled optimization.

We have provided you with starter code. All code for Part 1 is located in the directory `src/submission/goal_conditioned_rl` and all code for Part 2 is located in `src/submission/meta_rl`.

Submission

Submitting the Code and Experiment Runs: In order to turn in your code and experiment logs, run the `collect_submission.sh` script located in `src/`:

```
bash collect_submission.sh
```

1 Goal Conditioned Reinforcement Learning

In Part 1, we will be running goal-conditioned Q-learning on two problems:

1. a toy problem where we flip bits in a bit vector to match it to the current goal vector
2. move the end effector of a simulated robotic arm to the desired goal position

Applying hindsight relabeling to a goal-conditioned reinforcement learning setting is one of the most promising and commonly-used ways to improve sample efficiency and exploration challenges of RL!

Setup: Please navigate to `src/submission/goal_conditioned_rl` and follow the instructions in the `README.md`.

Part 1 Code Overview: The code consists of several files to enable Q-learning. All code for Part 1 is located in the directory `src/submission/goal_conditioned_rl`. You are expected to write the code in the following files: `run_episode.py`, `trainer.py`. A brief description for the codebase is provided here:

1. `trainer.py`: The main training loop. Alternates between collecting transitions using Q-value networks and training the networks on collected transitions.
2. `run_episode.py`: Collects an episode using the current Q-value network, and returns the transitions, collected reward and whether the agent was successful during the episode.
3. `replay_buffer.py`: Buffer for storing the transitions collected in the environment.
4. `q_network.py`: Creates a feedforward neural network with one hidden layer. This neural net represents our Q-network.
5. `bit_flip_env.py`: The source code for the bit flipping environment, which is set up to follow the gym API with an `__init__`, `reset` and `step` functions.
6. `sawyer_action_discretize.py`: Wraps the SawyerReachXYEnv from multiworld environment and converts the continuous action space into a discrete action space with a simplified 2D observation space.
7. `main.py`: Main file to configure the experiment.

Environments:

You will be running RL methods on two environments:

Environment 1: Bit Flipping Environment In the bit-flipping environment, the state is a binary vector with length n. The goal is to reach a known goal vector, which is also a binary vector with length n. At each step, we can flip a single bit in the vector (changing a 0 to 1 or a 1 to 0). This environment can very easily be solved without reinforcement learning, but we will use the DQN algorithm to understand how adding hindsight relabelling (HER) can improve performance. An example of this environment is shown in Table 1.

The bit flipping environment is an example of an environment with sparse rewards. At each step, we receive a reward of -1 when the goal and state vector do not match and a reward of 0 when they do. With a larger vector size, we receive fewer non-negative rewards.

Timestep	Current State	Action	Reward	Next State
1	[1,0,0,0,1,1]	0	-1	[0,0,0,0,1,1]
2	[0,0,0,0,1,1]	5	-1	[0,0,0,0,1,0]
3	[0,0,0,0,1,0]	3	0	[0,0,0,1,1,0]

Table 1: Illustration of an example bit flipping environment with $n = 6$. The initial state of the environment is [1,0,0,0,1,1] and the goal is to reach the state [0,0,0,1,1,0].

Environment 2: 2D Sawyer Arm (Figure 1)

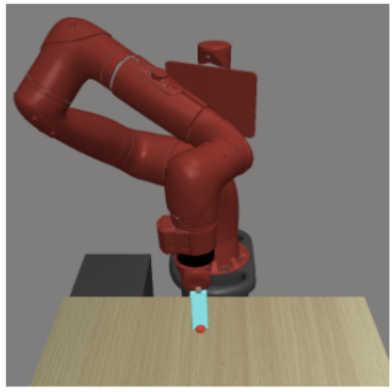


Figure 1: Illustration of an example Sawyer environment.

The Sawyer Arm (see Figure 1) is a multi-jointed robotic arm for grasping and reaching (see more details here). The arm operates in a 2D space, and the goal is to move the robot to a set of coordinates. The sawyer reach is an example of a dense reward environment, where the reward is given by negative Euclidean distance between the robot arm and the goal state. The end-effector (EE) is constrained to a 2-dimensional rectangle parallel to a table. The action controls EE position. The state is the XY position of the EE and the goal is an XY position of the EE.

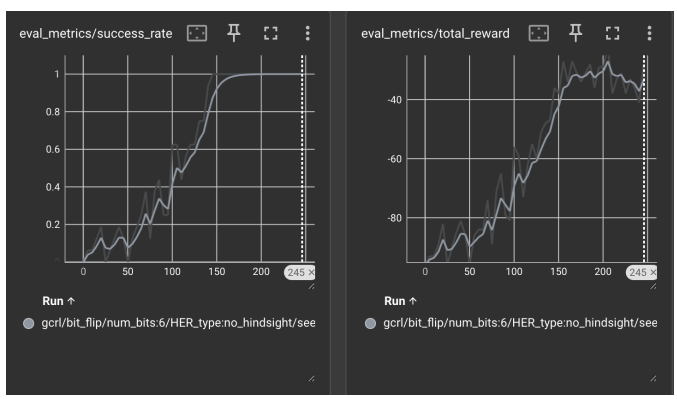
(a) **[11 points (Coding)] Implementing Goal Conditioned Reinforcement Learning**

We start this problem with a goal-conditioned implementation of DQN. The Q-function takes in the concatenated state and goal as input. You can think of the goal-conditioned implementation as an extended Markov decision process (MDP), where your state space contains both the original state and the goal. We will use this goal-conditioned Q-network to collect episodic data in `run_episode.py`.

For this part of the assignment, complete `run_episode.py` and run the following command:

```
python -m goal_conditioned_rl.main --env bit_flip --num_bits=6 --num_epochs=250 --
her_type no_hindsight
```

The evaluation metrics should be available in tensorboard events logged in `logs/` by default. **Verify** the `eval_metrics`, that is `total_reward` should be above -40.0 and success rate should be 1.0. This plot illustrates the performance without using HER.



(i) For simplicity, we will only consider the *greedy* action with respect to Q-network. Pass this action to `env.step`.

(ii) The `env.step` function returns `next_state`, `reward`, `done`, `info`, where `info` is a dictionary containing the a boolean under the key '`successful_this_state`', indicating whether the state was successful or not. Use this value to update succeeded, such that `run_episode` returns `True` if the *policy was successful at any step of the episode*. To understand more about `env.step`, read the documentation for the function in `bit_flip_env.py`.

(iii) Ensure that floating point numpy arrays use `np.float32`. You may need to recast some of the arrays to ensure that.

(b) **[6.50 points (Coding)] Adding HER to Bit Flipping**

With HER, the model is trained on the actual (state, goal, reward) tuples along with (state, goal, reward) tuples where the goal has been relabeled after the fact. The goals are relabeled to the state that was *actually reached* and the rewards should be relabeled correspondingly. In other words, we pretend that the state we reached was our goal all along. HER gives us more examples of actions that lead to positive rewards, effectively doubling our experience (since we use the original episode as well as the relabeled one). The reward function for relabeled goals is the same as the environment reward function; for the bit flipping environment, the reward is -1 if the state and goal vector do not match and 0 if they do match.

There are three different variations of HER: `final`, `random`, and `future`. Suppose the data collected in an episode consists of the following (state, goal, reward) tuples: $\{(s_t, g, r_t)\}_{t=0}^{T}$, where t indicates each time step in the episode. Given a (state, goal, reward) tuple (s_i, g_i, r_i) each variation of HER relabels the goal g_i differently:

- `final`: The final state of the episode is used as the goal.
- `random`: A random state in the episode is used as the goal.
- `future`: A random future state of the episode is used as the goal. Specifically, if you want to relabel the goal in the tuple (s_i, g_i, r_i), only states s_j with $j > i$ can be used.

Implementation Notes:

- Always use `copy()` when assigning an existing numpy array to a new variable. NumPy does not create a new copy by default.
- When choosing a new goal for relabelling, choose the state corresponding to `next_state` from the experience.

More details of these three implementations are in the comments of `trainer.py`. Implement the three variations of HER in the function `update_replay_buffer` in `trainer.py`. You **do not** need to submit a plot for this.

(c) **Analyzing HER for Bit Flipping Environment**

Once you have completed the previous problems, we can analyze the role of HER in goal-conditioned RL. We analyze the performance of HER as the size of the bit vector is increased from 6 to 25. For each of the parts (i) to (iv), submit a tensorboard screenshot showing the `eval_metrics` for different runs on the same plot (check the correct event files). Please make sure that each screenshot contains BOTH the eval/success_rate plot AND the eval/total_reward plot.

Run the following commands from inside the submission directory:

(i) **[1 point (Written)]**

```
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=6 --num_epochs=250 --
    her_type no_hindsight
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=6 --num_epochs=250 --
    her_type final
```

(ii) **[1 point (Written)]**

```
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --
    her_type no_hindsight
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --
    her_type final
```

(iii) **[1.50 points (Written)]**

```
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=25 --num_epochs=1000 --
    her_type no_hindsight
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=25 --num_epochs=1000 --
    her_type final
```

Referencing the images generated in part (i) to (iii), explain why you expect the methods to perform in the observed manner for the varying numbers of bits.

(d) **[1.50 points (Written)] Analyzing HER with Baseline**

Finally, we will compare the three versions of HER, with the baseline of not using HER:

```
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --her_type
    no_hindsight
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --her_type
    final
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --her_type
    random
python -m goal_conditioned_rl.main --env=bit_flip --num_bits=15 --num_epochs=500 --her_type
    future
```

Since two of the commands (`her_type no_hindsight` and `her_type final`) are identical to part (c), you **do not** need to run them again.

Explain your findings from these four runs and provide justification as to why you expect the methods to perform the way it did for the varying relabelling strategies.

(e) **Analyzing HER for Sawyer Reach**

If implemented correctly, HER should work for the second environment, Sawyer Reach.

Compare the performance of the Sawyer arm with and without HER. Run the following commands:

```
python -m goal_conditioned_rl.main --env=sawyer_reach --num_epochs=1000 --her_type
    no_hindsight
python -m goal_conditioned_rl.main --env=sawyer_reach --num_epochs=1000 --her_type final
```

(i) **[1.50 points (Written)]** Submit the tensorboard screenshot comparing the `eval_metrics` in your report. Please make sure that the screenshot contains BOTH the eval/success_rate plot AND the eval/-total_reward plot.

(ii) **[1.50 points (Written)]** Discuss your findings: Compare the role of HER in Bit Flipping Environment and Sawyer Reach. Comment on the differences between the contribution of HER, if any.

(f) **[1.50 points (Coding)] Bit Flipping Environment Logs**

For this question, please run our zip script to collect all your logs for the bit flipping environment with different number of bits and HER types. To verifiy if you submission is looking find, review and compare your folder structure with the one below:

```
submission
└── goal_conditioned_rl
    ├── *
    └── logs
        └── gcrl
            └── bit_flip
                ├── num_bits/6
                │   ├── HER_type/final
                │   │   └── seed/42 (or another number if you choose to use a different seed)
                │   │       └── YOUR_TF_FILE
                │   └── HER_type/no_hindsight
                │       └── seed/42 (or another number if you choose to use a different seed)
                │           └── YOUR_TF_FILE
                ├── num_bits/15
                │   └── Same as num_bits/6 but with runs for final/future/no_hindsight/random
                └── num_bits/25
                    └── Same as num_bits/6 but with runs for final/no_hindsight
```

```
│
└── sawyer_reach
    └── Same as num_bits/6 but with runs for final/no_hindsight
```

2 Meta-Reinforcement Learning

Setup: Please navigate to `src/submission/meta_rl` and follow the instructions in the README.

Code Overview: The main entry point for the code is via `src/submission/meta_rl/scripts/dream.py` and `src/submission/meta_rl/scripts/rl2.py`, which are the training scripts for DREAM and RL^2 respectively. Both of these can be invoked as follows:

```
$ python -m meta_rl.scripts.{script} {experiment_name} -b environment=\"map\"
```

In this invocation, {`script`} can either be `dream` or `rl2` and {`experiment_name`} can be any string with no white spaces. Results from this invocation are saved under `experiments/`{`experiment_name`}. For example, to launch the DREAM training script and save the results to `experiments/dream`, you would run:

```
$ python -m meta_rl.scripts.dream dream -b environment =\"map\"
```

To overwrite previous results, you would run:

You can pass the `--force_overwrite` or `-f` flag to run another experiment with the same experiment name, which will overwrite any previously saved files at the corresponding experiment directory. An example command for running DREAM is below:

```
$ python -m meta_rl.scripts.dream dream -b environment=\"map\" -f
```

If you do not pass this flag, the scripts will not allow you to run two experiments with the same experiment name.

There are two important subdirectories in each experiment directory:

- **Tensorboard:** Each experiment includes a `experiments/experiment_name/tensorboard` subdirectory, which will log important statistics about the training run, including the meta-testing returns under the `rewards/test` tag and the meta-training returns under the `rewards/training` tag. To view these, point Tensorboard at the appropriate directories. All curves are plotted with two versions, one where the x-axis is number of meta-training trials under `tensorboard/episode` and one where the x-axis is the number of environment steps `tensorboard/step`.
- **Visualizations:** Each experiment also includes a `experiments/experiment_name/visualize` subdirectory. This directory includes `.gif` videos of the agent during meta-testing and is structured as follows. The top level of subdirectories identify how many meta-training trials have elapsed before the video.

  Note that in the visualizations saved under `experiments/experiment_name/visualize`:

  - The agent is rendered as a red square.
  - The grid cells that the agent has visited in the episode are rendered as small origin squares.
  - There are four pairs of buses, rendered as blue, pink, cyan, and yellow squares.
  - The map is rendered as a black square.
  - The goal state is rendered as a green square, which obscures one of the buses.

  In experiments run with `dream.py`, the exploration episode is saved under {`video_num`}`-exploration.gif` and the exploration episode is saved under {`video_num`}`-exploitation.gif`. For example, the video under `experiments/dream/visualize/10000/0-exploration.gif` is the first exploration meta-testing episode after 10000 meta-training trials.

  In experiments run with `rl2.py`, {`video_num`}`.gif` contains both the exploration and exploitation episodes, with the exploration episode first. For example, the video under `experiments/rl2/visualize/10000/0.gif` contains the first exploration and exploitation episode after 10000 meta-training trials.

You will implement two short methods inside the `embed/encoder_decoder.py` file.

(a) **Grid World Navigation with Buses**

We consider a grid world illustrated in Figure 2. From a high level, the agent is given a goal each episode and must reach it in as few steps as possible. To quickly get to the goal, the agent may ride a bus. This brings the agent to the destination of that bus, which is the other bus of the same color. In different tasks, the buses in

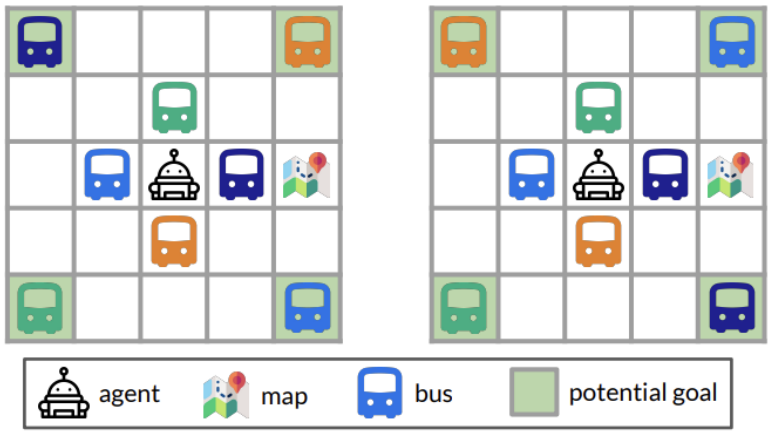


Figure 2: Two example tasks in the grid world domain. There are 4! different tasks, corresponding to different permutations of the buses in the corners.

the corners permute, while the buses in the center remain fixed. For example, in the left task in Figure 2, the center light blue bus's destination is the bottom right corner, while in the right task, its destination is the top right corner. Note that the goal is not part of the task, and all four corners are potential goals in all tasks. There is also a map at a fixed location in all of the tasks, which tells the agent the destination of each bus, when visited.

More concretely, the **state** consists of 4 components

- The agent's (x, y)-position in the grid
- A one-hot indicator of the object at the agent's current position (none, bus, map).
- A one-hot goal g corresponding to one of the four possible goal locations in the corners (shown in green).
- A one-hot that is equal to the problem ID μ (defined below) if the agent is standing on the map, and 0 otherwise. Standing on the map effectively encodes the destination of each bus.

The agent begins every episode at the center of the grid, as in Figure 2. During an episode, the goal is held fixed, while it is re-sampled uniformly across the 4 potential goal locations in each new episode.

At each timestep, the agent can take one of 5 **actions**:

- Move one cell up, down, left or right.
- Ride the bus that the agent is currently on. This teleports the agent to the other bus of the same color.

The agent receives $+1$ **reward** for reaching the correct goal position. The agent receives -0.3 **reward** at each timestep it is not at the correct goal, incentivizing it to reach the goal as quickly as possible. The episode ends if either the agent goes to any goal location (correct or incorrect) or if 20 timesteps have passed.

Each task is associated with a **problem ID** μ. The only thing that changes between tasks is the bus destinations: i.e., which colored bus appears in which outer corner. Therefore, there are $4! = 24$ different tasks. These tasks are **uniformly sampled** during meta-training and meta-testing.

Throughout the assignment, we consider the meta-RL setting with one *exploration episode* and one *exploitation episode*. The objective is to maximize the returns achieved in the exploitation episode, which we refer to as the *exploitation returns*. Note that the returns achieved in the exploration episode do not matter. During the exploitation episode, the agent is allowed to condition on the exploration episode $\tau^{\text{exp}} = (s_0, a_0, r_0, \ldots)$.

(i) **[1 point (Written)]** What returns are achieved by only taking the move action to get to the goal, without riding any buses: i.e., directly walking to the goal?

(ii) **[1 point (Written)]** If the bus destinations (i.e., the problem ID) were known, what is the optimal returns that could be achieved in a single exploitation episode? Describe an exploitation policy that

achieves such returns given knowledge of the bus destinations.

(iii) **[1 point (Written)]** Describe the exploration policy that discovers all of the bus destinations within the fewest number of timesteps.

(iv) **[1 point (Written)]** Given your answers in (ii) and (iii), what is the optimal exploitation returns achievable by a meta-RL agent?

(b) **End-to-End Meta-Reinforcement Learning**

In this problem, we'll analyze the performance of end-to-end meta-RL algorithms on the grid world. To do this, start by running the RL^2 agent on the grid world navigation task for 50,000 trials by running the below command. This should take approximately $2-3$ hours. Note that, by default, checkpoints of your models are saved every 20,000 iterations. Run inside the submission directory:

```
python -m meta_rl.scripts.rl2 rl2 -b environment=\"map\"
```

Examine the Tensorboard results under the tag `reward/test` in the `experiments/rl2` directory. Please take a screenshot and submit the tensorboard results of the `reward/test` graph.

(i) **[1.50 points (Written)]** Attach the screenshot of the graph here. To 1 decimal place, what is the average meta-testing exploitation returns RL^2 achieves after training?

(ii) **[1.50 points (Written)]** Examine the videos saved under `experiments/rl2/visualize/36000/`. Describe the exploration and exploitation behaviors that RL^2 learns. Does RL^2 achieve the optimal returns?

(c) **[3 points (Coding)] DREAM**

In part (a), we observed some shortcomings of end-to-end and existing decoupled meta-RL approaches. In this problem, we'll implement some components of DREAM, which attempts to address these shortcomings, given the assumption that each meta-training task is assigned a unique *problem ID* μ. During meta-testing, DREAM does not assume access to the problem ID.

From a high level, DREAM works by separately learning exploration and exploitation. To learn exploitation, DREAM learns an exploitation policy $\pi_\phi^{\text{exp}}(a \mid s, z)$ that tries to maximize returns during exploitation episodes, conditioned on a task encoding z. DREAM learns a *encoder* $F_\psi(z \mid \mu)$ to produce the task encoding z from the problem ID μ. Critically, this encoder is trained in such a way that z contains only the information necessary for the exploitation policy π_ϕ^{exp} to solve the task and achieve high returns.

By training the encoder in this way, DREAM can then learn to explore by trying to recover the information contained in z. To achieve this, DREAM learns an exploration policy π_ϕ^{exr}, which produces an exploration trajectory $\tau^{\text{exp}} = (s_0, a_0, r_0, \ldots)$ when rolled out during the exploration episode. To recover the information contained in z, DREAM fits a maximize the mutual information between the encoding z and the exploration trajectory τ^{exp}:

$$\max I(F_\psi(z \mid \mu), \tau^{\text{exp}}).$$

This objective can be optimized by maximizing the following variational lower bound:

$$\mathcal{J}(\omega, \phi) = \mathbb{E}_{\mu \sim P_\mu, \tau^{\text{exp}} \sim \pi_\phi^{\text{exr}}}[\log q_\omega(z \mid \tau^{\text{exp}})]$$

where $q_\omega(z \mid \tau^{\text{exp}})$ is a learned *decoder*. Note that this decoder enables us to convert an exploration trajectory τ^{exp} into a task encoding z that the exploitation policy uses. This is critical for meta-test time, where the problem ID is unavailable, and z cannot be computed via the encoder $F_\psi(z \mid \mu)$.

The objective $\mathcal{J}(\omega, \phi)$ is optimized with respect to both the decoder q_ω and the exploration policy π_ϕ^{exr}.

For simplicity, we parametrize the decoder $q_\omega(z \mid \tau^{\text{exp}})$ as a Gaussian $\mathcal{N}(g_\omega(\tau^{\text{exp}}), \sigma^2 I)$ centered at a learned $g_\omega(\tau^{\text{exp}})$ with unit variance. Then, $\log q_\omega(z \mid \tau^{\text{exp}})$ equals negative mean-squared error $-\|g_\omega(\tau^{\text{exp}}) - \text{stop-gradient}(z)\|_2^2$ plus some constants independent of ω. Overall, *maximizing* $\mathcal{J}(\omega, \phi)$ with respect to the decoder parameters ω is equal to *minimizing* the below mean-squared error **with respect to** ω:

$$\mathbb{E}_{z \sim F_\psi(\mu)}\left[\|g_\omega(\tau^{\text{exp}}) - \text{stop-gradient}(z)\|_2^2\right].$$

Code: Fill in the `.compute_losses` method of the `EncoderDecoder` in `encoder_decoder.py` to implement the above equation for optimize $\mathcal{J}(\omega, \phi)$ with respect to the decoder parameters ω.

(d) **[6 points (Coding)]** To optimize $\mathcal{J}(\omega, \phi)$ with respect to the exploration policy parameters ϕ, we expand out $\mathcal{J}(\omega, \phi)$ as a telescoping series:

$$\mathcal{J}(\omega, \phi) = \mathbb{E}_{\mu \sim P_\mu(\mu)}[\mathcal{J}[g_\omega(z \mid s_0)]] + \mathbb{E}_{\mu \sim P_\mu(\mu), \tau^{\text{exp}} \sim \pi_\phi^{\text{exr}}} \left[\sum_{t=0}^{T-1} \log q_\omega(z \mid \tau_{t+1}^{\text{exp}}) - \log q_\omega(z \mid \tau_t^{\text{exp}}) \right],$$

where τ_t^{exp} denotes the exploration trajectory up to timestep t: $(s_0, a_0, r_0, \ldots, s_t)$. Only the second term depends on the exploration policy, and since it occurs per timestep, it can be maximized by treating it as the following intrinsic reward function r_t^{int}, which we can maximize with standard reinforcement learning:

$$r_t^{\text{int}}(a_t, r_t, s_{t+1}, \tau_{t+1}^{\text{exr}}, \mu) = \mathbb{E}_{z \sim F_\psi(\mu)} \left[\log q_\omega(z \mid \tau_{t+1}^{\text{exr}}) - \log q_\omega(z \mid \tau_t^{\text{exp}}) \right]. \tag{1}$$

Note that τ_{t+1}^{exr} is equal to τ_t^{exp} with the additional observations of (a_t, r_t, s_{t+1}). The exploration policy is optimized to maximize this intrinsic reward r_t^{int} instead of the extrinsic rewards r_t, which will maximize the objective $\mathcal{J}(\omega, \phi)$. Intuitively, r_t^{int} is the "information gain" representing how much additional information about z – which encodes all the information to solve the task – the tuple s_{t+1}, r_t, a_t, s_t contains over what was already observed in τ_t^{exp}.

Code: Implement the reward function $r_t^{\text{int}}(a_t, r_t, s_{t+1}, \tau_{t+1}^{\text{exr}}, \mu)$ by filling in the `label_rewards` function of `EncoderDecoder` in `encoder_decoder.py`. Note that you'll need to make the same substitution for $\log q_\omega(z \mid \tau^{\text{exp}})$ in Equation (1) that we used in part a).

(e) Check your implementation by running DREAM inside the submission directory:

i. **[1.50 points (Written)]**

```
python -m meta_rl.scripts.dream dream -b environment=\"map\"
```

Submit the plot for test returns under the tag `rewards/test` from the `experiments/dream` directory. Submit the plot under `tensorboard/step`, not the plot under `tensorboard/episode`. If your implementation in part (c) and (d) is correct, you should see the test returns training curve improve within 30 minutes of training. By around 40 minutes, the test returns curve should begin to look different from RL^2. The total run should take around 2-3 hours.

ii. **[1.50 points (Written)]** Does DREAM achieve optimal returns in your results? Based on what you know about DREAM, do these results align with your expectations? Why or why not?

iii. **[2 points (Coding)]** Include your TensorBoard training logs from `experiments/dream/tensorboard/episode` in your submission. The autograder will parse these logs and verify that your DREAM implementation achieves a maximum test reward of at least 0.69 under the `reward/test` tag. You can run `collect_submission.sh` as needed to generate a zip with the relevant information.

(f) **[2 points (Written)]**

Inspect the videos saved under `experiments/dream/visualize/28000` or a later step after DREAM converges. Describe the exploration and exploitation behaviors that DREAM has learned.

References

[1] Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay. In *Neural Information Processing Systems (NeurIPS)*, 2017.

[2] Leslie Pack Kaelbling. Learning to achieve goals. In *International Joint Conferences on Artificial Intelligence (IJCAI)*, 1993.

[3] Yan Duan, John Schulman, Xi Chen, Peter L. Bartlett, Ilya Sutskever, and Pieter Abbeel. RL^2: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.

[4] Evan Liu, Aditi Raghunathan, Percy Liang, and Chelsea Finn. Explore then execute: Adapting without rewards via factorized meta-reinforcement learning. *arXiv preprint arXiv:2008.02790*, 2020.